### Exit a Loop with exitLoop

The keyword *exitLoop* exits a loop. Optionally, you can set the number of loops to be exited by specifying an *integer* number, for example `exitloop 2 // two loops`.

#### Examples

```
for var y := 1 to DataTable.yDim
   for var x := 1 to DataTable.xDim
       if DataTable[x,y] ~= "looking for this"
         print x, " ", y
         exitloop // exits the inner loop
      end
   next
next
```

```
for var y := 1 to DataTable.yDim
   for var x := 1 to DataTable.xDim
       if DataTable[x,y] ~= "looking for this"
         print x, " ", y
         exitloop 2 // exits the inner and outer loop
      end
   next
next
```

#### See also

**exitloop [SimTalk] - keyword**

## Suspending Methods

### Suspending Methods

The execution of a *Method* can be interrupted (suspended) for a number of reasons.

#### Remarks

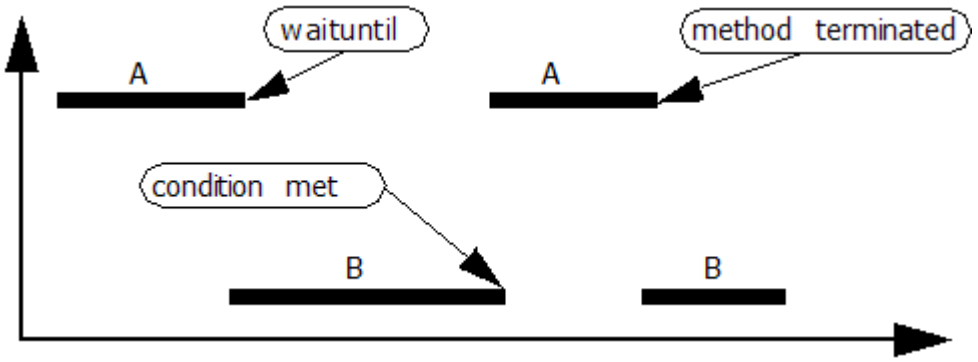
A *Method* might, for example, call another *Method*, or it may move a *MU* onto an object for which you programmed an **Entrance Control**. The execution of the *Method* will then be interrupted until the **Entrance Control** is executed, and will be continued once the **Entrance Control** is finished.

Another possibility would be to conditionally suspend the execution of a *Method*. It stops the execution until a certain condition is fulfilled. Other *Methods* can then be executed independent of the suspended *Method*. As soon as the condition is fulfilled, the execution of all *Methods* will be interrupted and the execution of the suspended *Method* will be continued.

The **waituntil-instruction** and the **stopuntil-instruction** allow you to suspend the execution of a method according to a condition you define. If this condition is not *true*, the interpreter saves the entire call chain, including parameters and local variables, and either starts to execute other *Methods* or continues the simulation run. As soon as the condition is *true*, the interpreter starts executing the suspended *Method* at the place in the source code at which it was suspended. An **observer** can watch this and initiate the appropriate action.

Let's illustrate this with an example:

The interpreter executes *MethodA* and comes across a `waituntil` instruction. The condition is not *true*, thus the interpreter suspends *MethodA*. Then, it executes *MethodB*, which changes the model during its execution, so that the condition for *MethodA* is *true*. At this time, the interpreter immediately interrupts *MethodB*, reactivates *MethodA* and executes it. Then, the interpreter resumes executing *MethodB*.

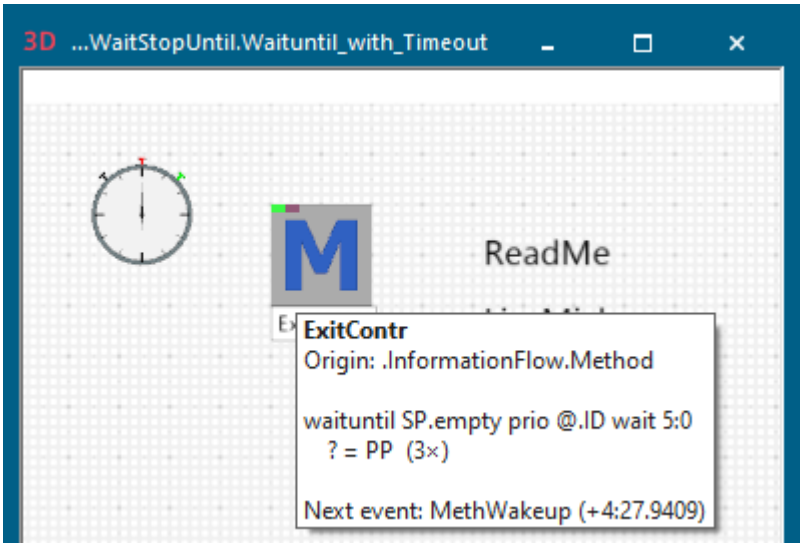



The `waituntil`-statement and the `stopuntil`-statement behave differently when several methods are to be activated at the same time. Then, it might happen, that after the *Method* with the highest priority has been executed, the condition for the other *Methods* takes the value *false* again.

- For *Methods* that were suspended with `waituntil` a new evaluation of the condition and the priorities for each additional *Method* to be woken up takes place after processing each *Method* that has been woken up.
- *Methods* that were suspended with `stopuntil` will be woken up in any case when the condition has been fulfilled once. This means that no new evaluation of the condition takes place.

If a *Method* is being executed during the **reset phase** and suspends itself with a **wait instruction**, *Plant Simulation* deletes the suspension at the end of the **reset phase**.

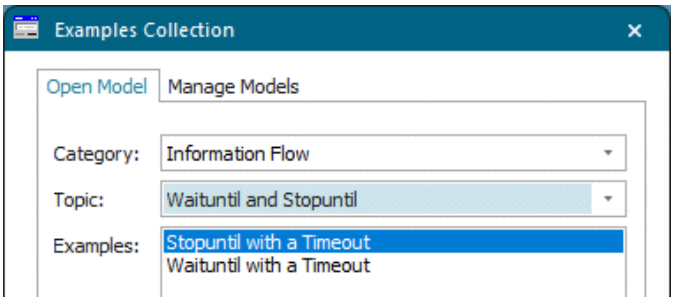
A *Method* that is suspended by **a waituntil-statement or by a stopuntil-statement**, by a **wait**-instruction, or by a **sleep**-instruction, shows a *tooltip* in the *Frame window*.

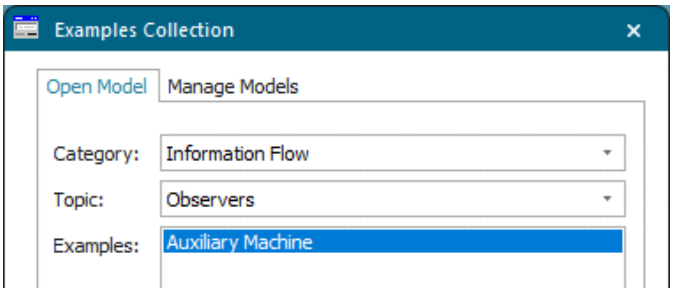



The *tooltip* shows:

- The **caller of the method**, in our case the *ExitContr*
- The **suspended instruction**, in our case `waituntil SP.empty prio @.ID wait 5:00 ? = PP`
- The **number of suspended methods**, in our case the *ExitContr* is suspended three times

Compare the sample models in the **Examples Collection**.




**See also**

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Interrupting Method Execution with Wait**

**Time Limitation for Waituntil and Stopuntil**

**Watchable Values**

**Create and Delete an Observer**

**Maximum Number of Suspended Methods [preferences]**

**Maximum Depth of Calls [preferences]**

**Maximum Number of Call Chains [preferences]**

## waituntil and stopuntil

### waituntil [SimTalk] and stopuntil [SimTalk]

The *waituntil/stopuntil-statements* suspend method execution until the condition set in the statement evaluates to *true*.

#### Remarks

The simulation continues running, and that other *Methods* can be executed during this time. As soon as the condition is fulfilled, the suspended *Method* will be woken up immediately and its execution will be continued. If *Methods* are being executed at the point in time at which the condition is fulfilled, their execution will be interrupted and will only then be continued after the *Method*, that has been woken up, has been executed all the way.

The *waituntil/stopuntil-statement* can also **observe** expressions for which only the last part of the path is **watchable**, for example `Station.Origin.Name`.

The statements consists of:

```
waituntil condition [prio number] [wait timespan:time]
stopuntil condition [prio number] [wait timespan:time]
```

- The keyword **waituntil** or **stopuntil** respectively

  **Note**

  You should never use *SimTime* in a method, which affects the simulation, with a **waituntil-instruction** or a **stopuntil-instruction**.

  The time does not advance continuously, but jumps from event to event. If you activate the animation, *Plant Simulation* generates additional events so that the **waituntil-instruction** or **stopuntil-instruction** might be woken up at an earlier point in time than intended.

- A condition (a *boolean* expression)
- The optional keyword **prio**, and an *integer* expression used to analyze the priority.
- The keyword **wait** if you want to set a time limit after which the statement will be woken up, although the condition has not been fulfilled yet. Use the keyword **waitExpired** to query if the time-span has elapsed or not.

**Note**

Do not use these statements within a formula. In a formula the interpreter cancels the execution of the *Method* with an error message.

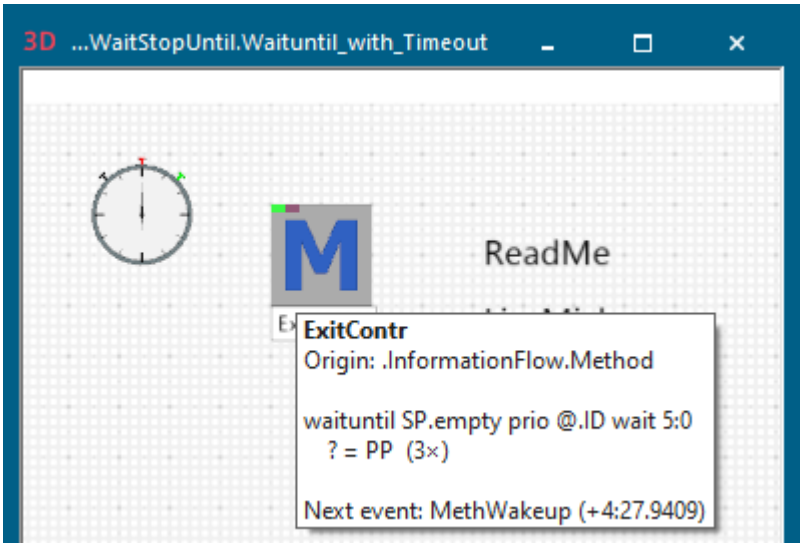
If you want to wake up several suspended *Methods* simultaneously, it might happen, that after the *Method* with the highest priority has been executed, the condition for the other *Methods* takes the value *false* again.

- For *Methods* that were suspended with `waituntil` a new evaluation of the condition and the priorities of each additional *Method* to be woken up takes place after processing each *Method* that has been woken up.
- *Methods* that were suspended with `stopuntil` will be woken up in any case when the condition has been fulfilled once. This means that no new evaluation of the condition takes place.

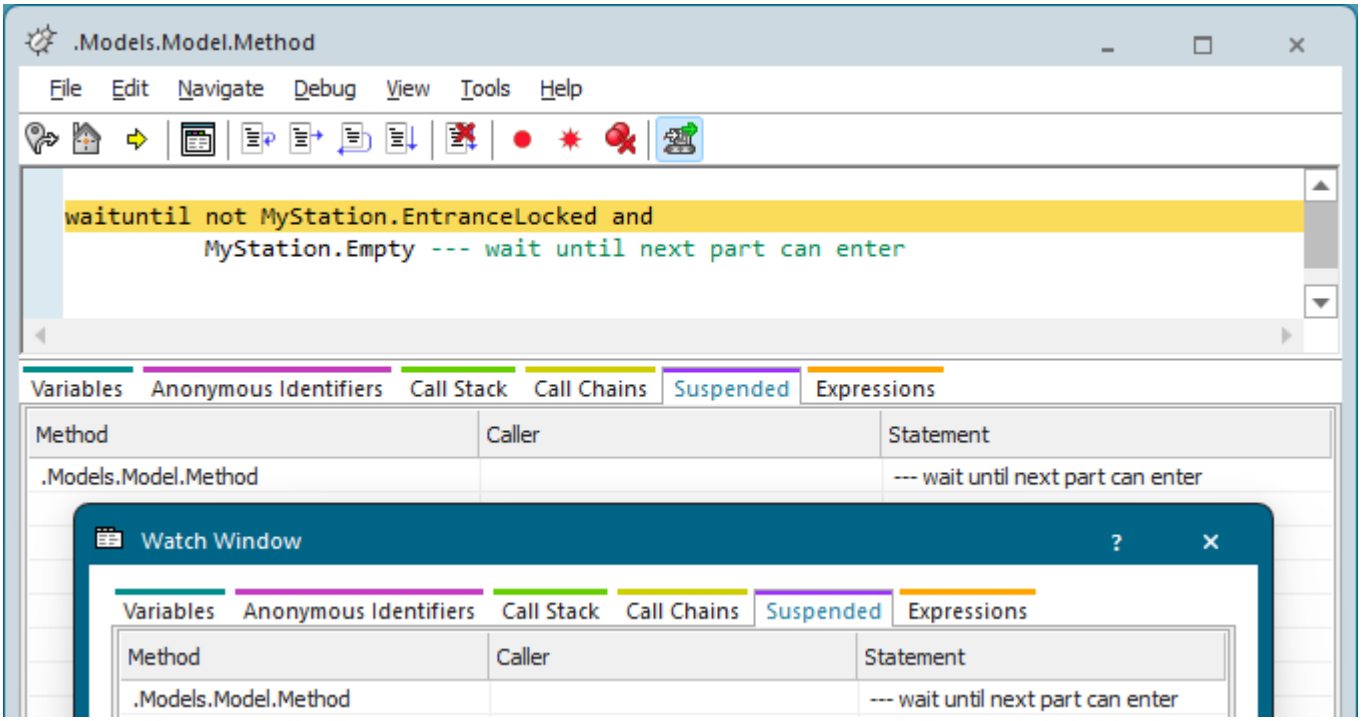
When you reset your simulation model, *Plant Simulation* deletes suspensions of *Methods*, which are located in the same *Frame* as the *EventController* or in a *sub-Frame* of this *Frame*. It also deletes suspensions of *Methods*, whose caller (the anonymous identifier **?**) is located in the same *Frame* as the *EventController* or in a *sub-Frame* of this *Frame*. This means that when you reset an *EventController*, *Plant Simulation* no longer deletes suspensions of *Methods* of other simulation models within the model file.

The *waituntil/stopuntil-statement* are not event-based, so no *EventController* is required, as compared to the **wait**-statement, which does require an *EventController*.

A *Method* that is **suspended** by a **waituntil-statement** or by a **stopuntil-statement** shows this in the *tooltip* in the *Frame window*.



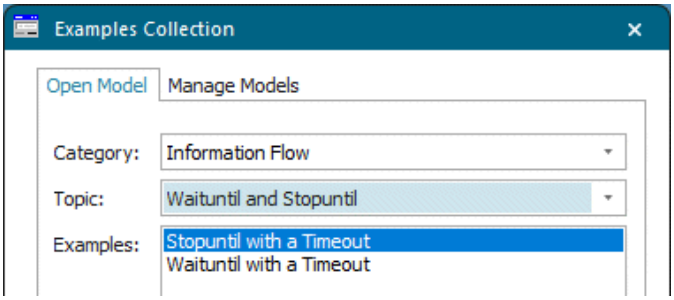

If you add a comment after a *waituntil-statement*, a *stopuntil-statement*, a *wait-statement*, or a *sleep-statement* to the source code of your *Method* and if this comment starts with three hyphens `---` or with three forward slashes `///`, *Plant Simulation* does not show the entire statement on the **Tab Suspended** of the **Watch Window** and of the *Method Debugger*, but only the comment.

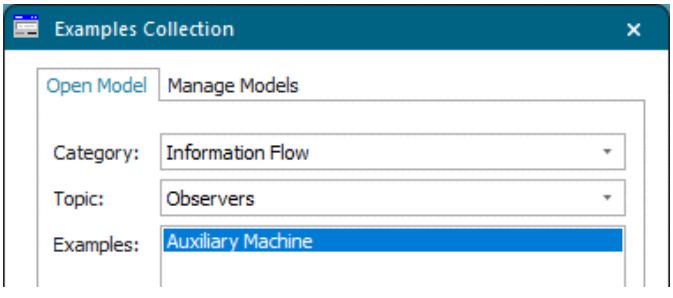



The method **deleteSuspendedMethods** deletes all suspensions of all methods.

*Plant Simulation* shows a suspended *Method* with a purple rectangle on the top border of the icon.


Compare the sample models in the **Examples Collection**.




**See also**

**SimTime [SimTalk]**

**Condition [SimTalk]**

**Priority [method execution]**

**Waking up Methods**

**Simultaneously Waking up Several Methods**

**Create and Delete an Observer**

**Simultaneously Waking up Several Methods with waituntil**

**Simultaneously Waking up Several Methods with stopuntil**

**Maximum Number of Suspended Methods [preferences]**

**Video on YouTube**

**https://youtu.be/5t-wLNmKpbU?si=0c4xB49lJGLxb_2L&t=862**

## Condition [SimTalk]

The expression that you enter into *waituntil/stopuntil-statements* determines under which condition the interpreter continues executing the *Method*.

### Remarks

If the condition is *true*, the interpreter continues the execution with the next statement.

If the condition is *false*, the interpreter suspends the *Method* and monitors the individual components of the expression. If one component changes, the interpreter restarts the *Method*, so that the condition can be analyzed again.

Let's illustrate this with a example:

A *Transporter* is to load a *MU* and to drive on afterward. However, we do not know if the *Transporter* or the *MU* arrives at the station first. To solve this problem, we can either enter **Exit Controls** into the *Transporter* and the *MU*, checking if the other one is at the station or not, or we can use a single **Exit Control** for the *Transporter* combined with a **waituntil-statement**:

```
waituntil MU_place.occupied prio 1
MU_place.Cont.move(@)
```

By using the **waituntil-statement**, the **Exit Control** makes sure that a *MU* is available to be loaded. If no *MU* is available, the interpreter suspends the *Method* until a *MU* arrives, and activates the *Method* as soon as the *MU* enters the location `MU_place`.

As the interpreter has to automatically break up the expression into components that it can observe:

- You can use the basic operators (+, –, *, /), comparison operators (=, <=, >=, /=), logical operators (`AND`, `OR`, `NOT`), and parentheses `( )`.
- You cannot use *Method* calls, access to tables, and built-in methods with parameters.
- You can, but should not, use built-in methods that produce side effects, such as `stack.pop` because the condition may have to be analyzed frequently.

*Plant Simulation* always evaluates logical operations from left to right. The evaluation terminates as soon as the value of the expression is certain.

### Example

```
waituntil Station.Cont /= void AND Station.Cont.Finished
```

As long as `Station.Cont` has the value `void`, it is already clear that the entire condition will be *false*, and therefore `Station.Cont.Finished` will not be evaluated and thus does not lead to a runtime error.

**Note**

If you are watching a *user-defined attribute* of data type *object*, its value can become `void` for two reasons:

1. A *Method* assigns the value `void` to the *user-defined attribute*. Then the condition will be re-evaluated and the *waituntil/stopuntil statement* might wake up.
2. The object to which the *user-defined attribute* points is deleted. Then the condition will not be re-evaluated and the *waituntil/stopuntil statement* will not wake up.

**See also**

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Priority [method execution]**

**Waking up Methods**

**Simultaneously Waking up Several Methods**

**Create and Delete an Observer**

**Simultaneously Waking up Several Methods with waituntil**

**Simultaneously Waking up Several Methods with stopuntil**

**Maximum Number of Suspended Methods [preferences]**

## Priority [method execution]

*Plant Simulation* may suspend several *Methods* based on identical or similar conditions. Consequently, a set of several *Methods* may be woken up at the same time. In this case the interpreter wakes up the *Method* with the highest priority and continues executing it.

### Remarks

After *Plant Simulation* finished executing the *Method* with the highest priority or if it comes across another **waituntil statement** with an unfulfilled condition during its execution, the interpreter re-analyzes the conditions of the remaining *Methods* in the set. This yields a new subset of *Methods* to be woken up. If any of the conditions are still *true*, the interpreter re-analyzes the priorities and again selects the *Method* with the highest priority.

For evaluating the priority you can use *Methods*, *DataTables*, or *parameters*. Avoid *side effects*, such as deleting *MUs* or changing a *global variable*, as the frequency and time of evaluation of the priority expression depend on both the interpreter and the state of the model.

### See also

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Condition [SimTalk]**

**Waking up Methods**

**Simultaneously Waking up Several Methods**

**Create and Delete an Observer**

**Simultaneously Waking up Several Methods with waituntil**

**Simultaneously Waking up Several Methods with stopuntil**

**Maximum Number of Suspended Methods [preferences]**

## Interrupting Method Execution with Wait

Interrupt the execution of a method with the keyword `wait` for the specified number of seconds of simulation time. The execution of the *Method* continues as soon as the time of the *EventController* advanced by the amount of time you specified for the wait statement.

### Syntax

```
wait timespan:real
```

### Remarks

The keyword **wait** interrupts the execution of a call chain for the number of seconds passed as the parameter of data type *real*. The simulation continues as if the call chain had been finished. *Plant Simulation* enters a **MethWakeup** event into the **List** of scheduled events in the *EventController* to continue executing the call chain after the time you specified has elapsed.

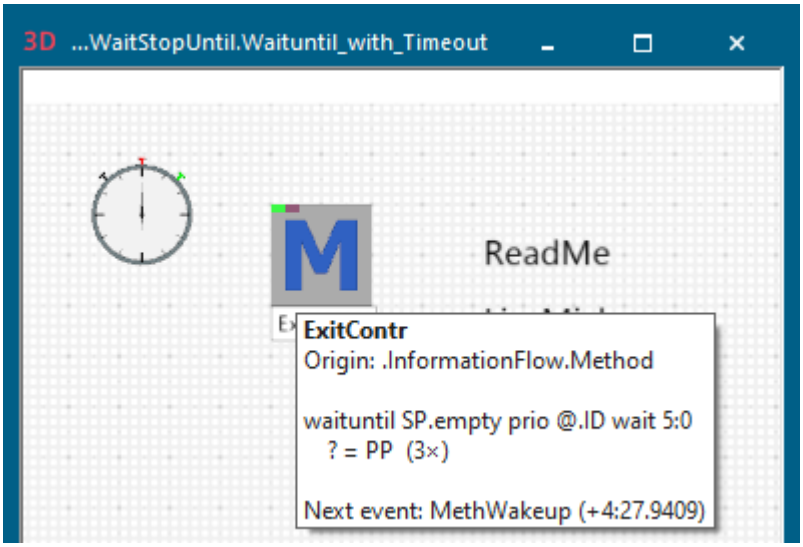
> **Note**
> This only works if your simulation model contains an *EventController*.

If a control is being executed during the **reset phase** and suspends itself with a **wait-instruction**, *Plant Simulation* deletes the suspension at the end of the **reset phase**.

You can also suspend a *Method class*, meaning a *Method* that is located in a folder in the *Class Library*, with a **wait-instruction** if the *Method class* was called by a *Method* that is located in a model containing an *EventController*.

*Plant Simulation* shows the state as a colored rectangle at the top border of the image.


A *Method* that is **suspended** by a **wait-instruction** shows this in the *tooltip* in the *Frame window*.




**Example**

```
var MU: object
repeat
    if not Station.occupied
       MU := .MUs.Container.create(Station)
    end
    wait 120
    if MU /= void
       mu.delete
    end
    wait 30
until false
```

### Time Limitation for Waituntil and Stopuntil

In *SimTalk 2.0* you can specify a time limitation for **waituntil** and **stopuntil statements**.

#### Remarks

If the **waituntil/stopuntil-statement** is suspended for this time duration it will be reactivated, although the condition has not yet been met.

The keyword **waitExpired** is set to *true* once the instruction is reactivated caused by the time limitation. If the suspension terminates because of the condition on the other hand, *waitExpired* is set to *false*.

```
waituntil Station.Empty wait 60
   if waitExpired
       [Station]
   else
       @.delete
   end
```

#### See also

**waituntil [SimTalk] and stopuntil [SimTalk]**

### Reactivating Methods

### Waking up Methods

The value of a condition causing a *Method* to be suspended can change because of a simulation event (for example a *MU* exiting the station) or method processing (for example, assignment of a value).

#### Remarks

If a *Method* is active, the interpreter interrupts the execution of the active *Method* and saves the entire call chain of the *Method*.

After that, the interpreter re-analyzes the conditions of the suspended *Methods*. The interpreter does not analyze conditions that do not depend on the change again.

If the condition is not *true*, it suspends the *Methods* immediately.

If the condition is *true*, the interpreter analyzes its priority and builds a priority list. It then selects the *Method* with the highest priority and resumes executing that *Method*, meaning it reactivates it. Method execution continues until *Plant Simulation* executed the entire active call chain, if an error occurs during execution or if a *Method* encounters a *waituntil-statement* again, whose condition is not met and the *Method* is thus suspended again.

**See also**

**Simultaneously Waking up Several Methods**

**Simultaneously Waking up Several Methods with waituntil**

**Simultaneously Waking up Several Methods with stopuntil**

**Priority [method execution]**

## Simultaneously Waking up Several Methods

*Plant Simulation* reactivates the suspended *Method* with the highest priority first if several *Methods* can be reactivated at the same time.

### Remarks

Waking up a *Method* may affect the conditions of the remaining *Methods*. Thus, some of the initially *true* conditions may become *false* after the first *Method* is executed.

For *Methods* that were suspended with **waituntil**, the interpreter analyzes the conditions again after the first reactivated *Method* is finished. Consequently, some *Methods* may remain suspended even though their condition was initially *true*.

For *Methods* that were suspended with **stopuntil**, the interpreter does not analyze the conditions a second time. The fact that a condition became *true* initially suffices for all remaining *Methods* to be reactivated. The interpreter reactivates the *Methods* according to the priority you specified.

### See also

**Simultaneously Waking up Several Methods with waituntil**

**Simultaneously Waking up Several Methods with stopuntil**

**Priority [method execution]**

## Simultaneously Waking up Several Methods with waituntil

Use `waituntil` to simultaneously wake up several *Methods*.

### Remarks

Suppose that *Transporters* are to drive from several objects to the same *Station*. Remember that a *Station* can only accept a *MU*, if it is **Empty**.

The source code of the **Exit Controls** of the predecessor objects looks like this:

```
waituntil Station.Empty prio @.Capacity
@.move(Station)
```

Suppose we have three suspended **Exit Controls**. Once the *Station* is empty, the *Transporter* with the highest **Capacity** drives to the *Station*, as it has the highest priority. After the *Transporter* drives to the *Station*, the condition `Station.Empty` is *false*. Thus, the remaining two *Transporters* stay where they are at and wait until the *Station* is empty again.

### See also

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Simultaneously Waking up Several Methods with stopuntil**

**Priority [method execution]**

## Simultaneously Waking up Several Methods with stopuntil

Use `stopuntil` to simultaneously wake up several *Methods*.

### Remarks

Suppose we model a *Store* using the *Variable* named *GateOpen*. *MUs* can only enter the *Store* if `GateOpen` is *true*. After the *MUs* move through the gate at the entrance of the *Store*, the condition `GateOpen` is *false* again.

The source code of the **Exit Control** of the predecessor objects like this:

```
stopuntil GateOpen prio 1
@.move(Store)
GateOpen := false
```

If `GateOpen` is *true*, the interpreter wakes up all suspended *Methods*, although `GateOpen` (and the suspension condition) is set to *false* after the first *Method* is called.

### See also

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Simultaneously Waking up Several Methods with waituntil**

**Priority [method execution]** during method execution

## Watchable Values

A watchable value is a value, which *Plant Simulation* can observe while it executes *SimTalk* code.

### Remarks

*Plant Simulation* can **watch** the conditions used in a **waituntil-instruction** or a **stopuntil-instruction** with an observer. A condition is **watchable** if all components of the *boolean* expression are watchable.

*SimTalk* only evaluates the condition anew if one of its watchable components changed during the simulation. Some examples of conditions that cannot be watched are the return values of *user-defined methods* and values that change continuously, such as the utilization of stations.

If the condition contains a component that *Plant Simulation* cannot watch, the *Method Debugger* will open and display the error message **Expression cannot be watched**.

The column **Watchable** in the window **Show Attributes and Methods** shows all *attributes*, *methods*, and *read-only attributes* that *Plant Simulation* can watch.


| Name | Value | Inherited | Watchable | Signature |
|---|---|---|---|---|
| AutomaticProcessing | true | ✔ | * | boolean |
| AutomaticSetup | true | ✔ | * | boolean |
| Capacity | 1 | ✔ | * | integer |
| Cont | VOID | | * | -> object |
| Empty | true | | * | -> boolean |
| EnergyCurrentState | "Operational" | | * | string |
| EnergyTargetState | "Operational" | | * | string |
| EntranceFree | true | | * | -> boolean |
| EntranceLocked | false | ✔ | * | boolean |
| ExitLocked | false | ✔ | * | boolean |
| Failed | false | | * | boolean |
| FailureActive | true | ✔ | * | boolean |
| Full | false | | * | -> boolean |
| FwBlockListEntry1 | VOID | | * | -> object |
| Label | "" | ✔ | * | string |
| Location | .Models.Model | | * | -> object |
| Name | "Station" | ✔ | * | string |
| NumMU | 0 | | * | -> integer |
| NumMUParts | 0 | | * | -> integer |
| Occupied | false | | * | -> boolean |
| Operational | true | | * | -> boolean |
| Pause | false | | * | boolean |
| PowerInput | 0.0 | | * | -> real |
| RemainingSetupTime | -1.0000 | | * | -> time |
| ResBlocked | false | | * | -> boolean |
| ResCurrentState | "Waiting" | | * | -> string |
| ResourceType | "Production" | ✔ | * | string |
| ResSetup | false | | * | -> boolean |
| ResWaiting | true | | * | -> boolean |
| ResWorking | false | | * | -> boolean |
| SetupOnlyWhenEmpty | false | ✔ | * | boolean |
| StatisticsStartRes | 0.0000 | | * | -> time |
| StatMaxNumMU | 0 | | * | -> integer |
| StatNumIn | 0 | | * | -> integer |
| StatNumOut | 0 | | * | -> integer |
| Stopped | false | | * | boolean |
| Unplanned | false | | * | boolean |
| ~ | .Models.Model | | * | -> object |

**See also**

**waituntil [SimTalk] and stopuntil [SimTalk]**

**Edit Observers [general description]**

### Create and Delete an Observer

**Video on YouTube**

**https://youtu.be/5t-wLNmKpbU?si=iskEWwLFhy6WvVlb&t=1088**

**https://youtu.be/9g6Uou-8eMc?si=tMGvgn6iYeolw7V8&t=318**